Agentic AI: Zero-to-Pro Roadmap

Nov 1, 2026 · @Priyam

Overview

An agent is an LLM running in a loop that uses tools, and this roadmap takes you from Python basics to production agents in about 6 months at 10 hours a week.

The Claude Agent SDK describes the loop as four repeating steps: gather context, take action, verify the work, repeat. Frameworks change every few months, but this loop, evaluation and safety stay stable. That is why you build an agent from scratch before touching any framework.

Stage	Topic	Time	Project
0	Foundations	2–3 weeks	none
1	LLM fundamentals	2–3 weeks	none
2	Core agent concepts	3–4 weeks	Project 1: CLI agent
3	RAG and context engineering	3 weeks	Project 2: document chat agent
4	Frameworks	4 weeks	Project 3: research agent
5	MCP and tool ecosystem	2–3 weeks	Project 4: MCP server
6	Evals, observability, safety	3 weeks	none
7	Production engineering	4 weeks	Project 5: deployed agent
8	Specialize and portfolio	ongoing	Capstone

YouTube links change, so resources are listed by channel and title for you to search. The one linked playlist (Karpathy) was verified. Tick the boxes as you go.

Stage 0: Foundations (2–3 weeks)

Agents spend most of their time calling APIs, so you need solid Python, async code and API habits before anything else.

- [ ] Python: functions, classes, type hints, virtual environments
- [ ] `async` / `await` (agents mostly wait on API calls)
- [ ] HTTP, REST, JSON and reading API docs

- [ ] Git, GitHub and the terminal
- [ ] Environment variables and secret handling (never commit API keys)

YouTube: freeCodeCamp has a course that starts from Python fundamentals and builds up to AI agent development. For Git, search "freeCodeCamp Git and GitHub for beginners".

Stage 1: LLM fundamentals (2–3 weeks)

Learn what an LLM does with your input and how to call one directly, with no framework in between.

- [ ] Tokens, context windows, temperature, and how transformers work (intuition only)
- [ ] Prompting: system prompts, few-shot examples, role and format control
- [ ] Structured outputs (JSON schemas) and streaming
- [ ] Calling an LLM API directly
- [ ] Cost and latency basics (tokens in vs out, caching)

YouTube:

- 3Blue1Brown's neural network and transformer series (search "3Blue1Brown neural networks").
- Andrej Karpathy's "Intro to Large Language Models" and "Deep Dive into LLMs like ChatGPT".
- Optional depth: Karpathy's Neural Networks: Zero to Hero, about 19 hours and math-heavy. Skip it at first and return later.

Stage 2: Core agent concepts (3–4 weeks)

This is the most important stage: you build a working agent loop yourself, with tools, before using any framework.

- [ ] Tool use / function calling: define a tool, parse the call, return the result
- [ ] The agent loop and when to stop it (max steps, budgets)
- [ ] ReAct (reason + act), planning and reflection
- [ ] Workflows vs. agents, and the patterns: prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer
- [ ] Short-term memory (conversation state) vs. long-term memory (stored facts)
- [ ] Error handling: retries, tool failures, malformed outputs
- [ ] **Project 1:** a CLI agent with 3 tools (web search, file read/write, calculator), built with no framework

YouTube:

- freeCodeCamp's course on building an AI coding agent in Python is a good template for Project 1.
- freeCodeCamp's "AI Agents For Beginners" (July 2026, about 3 hours) traces an agent's loop and how it manages session state.
- freeCodeCamp's "Building Agentic AI Workloads – Crash Course" (Jan 2026) covers the production side.

Read: Anthropic's "Building effective agents" post. It is text, not video, but it is the clearest statement of the patterns above.

Stage 3: RAG and context engineering (3 weeks)

Agents are only as good as the context you give them, so learn retrieval and how to manage a growing history.

- [ ] Embeddings, chunking strategies, vector databases (Chroma, pgvector, Qdrant)
- [ ] Hybrid search (keyword + vector) and reranking
- [ ] Agentic RAG: the agent decides when and what to retrieve
- [ ] Context management: summarizing, trimming and compacting long histories
- [ ] **Project 2:** a "chat with your documents" agent that cites its sources

YouTube: search "freeCodeCamp RAG course" and "James Briggs RAG".

Stage 4: Frameworks (4 weeks)

Learn one graph framework and one vendor SDK, not everything on the market.

- [ ] LangGraph: state, nodes, edges, conditional routing, checkpointing, human-in-the-loop
- [ ] Claude Agent SDK and/or the OpenAI Agents SDK
- [ ] Multi-agent patterns: handoffs, supervisor/worker, parallel sub-agents
- [ ] **Project 3:** a multi-step research agent (plan → search → write → critique)

YouTube:

- James Briggs: LangGraph 101, on building agent graphs and how LangGraph differs from LangChain.
- James Briggs: LangGraph Deep Dive, which builds a research agent.
- Sam Witteveen: LangGraph crash course with code examples.
- Harish Neel: LangGraph crash course covering agentic patterns, RAG and multi-agent systems.

- Traversy Media: a crash course on the Claude Agent SDK, which embeds Claude Code's capabilities directly into your own apps.

Stage 5: MCP and the tool ecosystem (2–3 weeks)

The Model Context Protocol is the standard way to plug tools and data into agents, so you should be able to build and consume MCP servers.

- [ ] MCP architecture: host, client, server
- [ ] The three primitives: tools, resources, prompts
- [ ] Build an MCP server (Python or TypeScript)
- [ ] Connect an MCP server to an agent
- [ ] Auth, permissions and trusting third-party servers
- [ ] **Project 4:** an MCP server wrapping an API or database you care about

YouTube:

- Web Dev Simplified: a crash course that builds an MCP server and client from scratch with the TypeScript SDK.
- Sam Witteveen: a walkthrough of the protocol's architecture and setup.
- Kubesimplify: a free course from the basics through building your own MCP servers.
- Matt Palmer: "Run MCP in 3 minutes", a quick overview.

Stage 6: Evals, observability and safety (3 weeks)

This stage separates professionals from hobbyists: without evals you cannot tell whether a change made your agent better or worse.

- [ ] Build an eval dataset: golden cases plus failures you have seen
- [ ] Deterministic checks, LLM-as-judge, and trajectory (step-by-step) evals
- [ ] Trace every run (LangSmith, Langfuse, Arize Phoenix or similar)
- [ ] Prompt injection, data exfiltration and tool-permission design
- [ ] Sandbox code execution and give tools least-privilege access
- [ ] Human approval gates for irreversible actions
- [ ] Cost caps and loop limits

Resources:

- YouTube: LangChain's "Building Better AI Agents: Observability and Evaluation" (Feb 2026, 47 min).

- DeepLearning.AI's Evaluating AI Agents, a free short course of about 2 hours 16 minutes. It is on their platform, not YouTube.
- Google and Kaggle's AI Agents Intensive includes a white paper on agent quality, from evaluation to observability. Search for the recordings.

Stage 7: Production engineering (4 weeks)

A demo that works once is not an agent product; this stage covers serving, reliability and cost.

- [ ] Serve agents with FastAPI, with streaming responses
- [ ] Background jobs and queues for long-running tasks
- [ ] Durable execution and resuming after failures (checkpointing, Temporal-style)
- [ ] Docker and cloud deployment
- [ ] Caching, rate limits and model routing (a small model for easy steps, a larger one for hard ones)
- [ ] Monitor cost, latency and failure rate in production
- [ ] **Project 5:** deploy one earlier project with tracing, evals in CI, and a cost dashboard

YouTube: search "FastAPI full course freeCodeCamp" and "Docker for beginners". Revisit the freeCodeCamp production crash course from Stage 2.

Stage 8: Specialize and build a portfolio (ongoing)

Pick one or two specializations, then ship public work that proves you can build and evaluate agents.

Specializations (choose one or two):

- [ ] Coding agents (sandboxes, diffs, test-driven loops)
- [ ] Browser and computer-use agents
- [ ] Data and analytics agents (SQL, notebooks)
- [ ] Voice and real-time agents
- [ ] Enterprise workflow agents (tickets, CRM, internal tools)

Capstone checklist:

- [ ] 3 public GitHub repos with clear READMEs, architecture diagrams, and an eval report in each
- [ ] 1 deployed, usable demo
- [ ] 1 write-up on what failed and how you fixed it (hiring managers value this)

- [ ] 1 open-source contribution (a framework, an MCP server, or docs)

Advice from experience

1. **Build from scratch first.** If you cannot write the loop yourself, you will not be able to debug a framework when it misbehaves.
2. **Start with workflows, and add autonomy only when you need it.** Many production "agents" are better as fixed pipelines with one or two decision points.
3. **Write evals early.** Most beginners skip them and then cannot tell whether a change helped.
4. **Be careful with "full course in 2 hours" videos.** Many are no-code or aimed at selling something. Prefer ones where you write code and the repo is public.
5. **The tools move fast.** Check the date on any tutorial and compare it against the framework's current docs before copying code.

Sources

Researched 2026-11-01. YouTube titles were confirmed through web search; check each video's date and description before committing time to it.

- freeCodeCamp: How to Build Agentic AI Workflows
- freeCodeCamp: Build an AI Coding Agent in Python
- Class Central: LangGraph 101 (James Briggs)
- Class Central: LangGraph Deep Dive (James Briggs)
- Karpathy: Neural Networks: Zero to Hero
- Kubesimplify MCP course
- DeepLearning.AI: Evaluating AI Agents
- DataCamp: Claude Agent SDK tutorial